Learning Language-Specific Layers for Multilingual Machine Translation

Telmo Pessoa Pires **Robin M. Schmidt** **Yi-Hsiu Liao** **Stephan Peitz**
Apple
{telmo, robin_schmidt, yihsiu_liao, speitz}@apple.com

arXiv:2305.02665v1 [cs.CL] 4 May 2023

Abstract

Multilingual Machine Translation promises to improve translation quality between non-English languages. This is advantageous for several reasons, namely lower latency (no need to translate twice), and reduced error cascades (e.g., avoiding losing gender and formality information when translating through English). On the downside, adding more languages reduces model capacity per language, which is usually countered by increasing the overall model size, making training harder and inference slower. In this work, we introduce Language-Specific Transformer Layers (LSLs), which allow us to increase model capacity, while keeping the amount of computation and the number of parameters used in the forward pass constant. The key idea is to have some layers of the encoder be source or target language-specific, while keeping the remaining layers shared. We study the best way to place these layers using a neural architecture search inspired approach, and achieve an improvement of 1.3 CHRF (1.5 SPBLEU) points over not using LSLs on a separate decoder architecture, and 1.9 CHRF (2.2 SPBLEU) on a shared decoder one.

1 Introduction

Multilingual Neural Machine Translation (MNMT) has received much attention from the Machine Translation community in recent years (Johnson et al., 2017; Aharoni et al., 2019; Freitag and Firat, 2020; Zhang et al., 2020; Fan et al., 2021; Yang et al., 2021; Tran et al., 2021). This interest is based on the many advantages it provides:

Scalability Instead of having one model per language pair, a single model suffices, significantly reducing maintenance efforts as well as the combined model size across all languages.

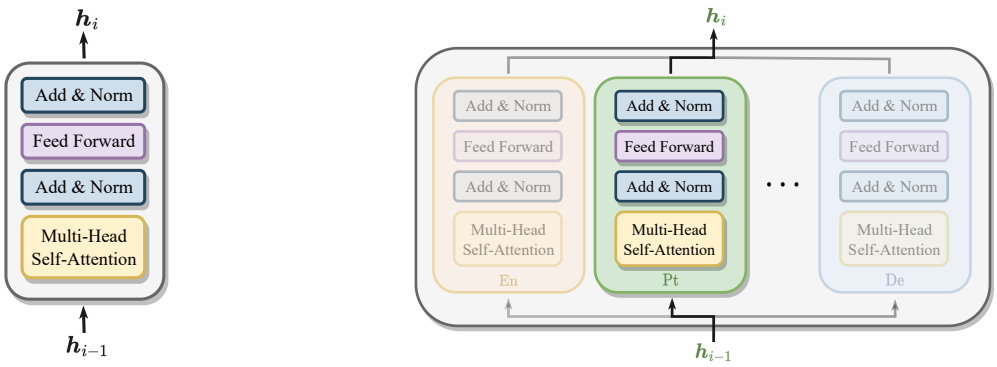
Inference Speed and Less Error Cascading Due to the availability of data, most production systems are English-centric, meaning translation between two non-English languages naïvely involves translating twice (i.e. pivoting), once to English, and once from English. This approach increases latency and contributes to error cascades, since the translation to or from English usually implies information loss, e.g. missing gender or formality distinctions that do not exist similarly in English.

Low-Resource Improvements Having a single model capable of handling multiple languages, means it can generalize across language boundaries and utilize characteristics of closely related translation directions to improve the translation quality for low-resource language-pairs (i.e. knowledge transfer). Although achieving good zero-shot translation quality remains a challenging task, MNMT has been shown to help (Johnson et al., 2017).

Despite the above advantages, training high quality multilingual models is a challenging task: as more languages are added, the more they compete for the model's parameters (Sachan and Neubig, 2018). A common solution is to increase the model size, but blindly doing so comes with its own troubles, as training becomes harder, inference slower, and the storage requirements increase, which makes them challenging to deploy to portable devices.

In this work, our goal is to increase the model capacity per language pair, while at the same time, letting the model share knowledge between languages, and without increasing the inference cost. To this end, and combined with the observation from Kudugunta et al. (2019) that the translation process in Transformer models starts in the top encoder layers, we propose an architecture with shared and language-specific weights. Figure 2 shows one such architecture, where layers 3 and 4[1] are source language-specific, layers 13, 14, and 15 are target language-specific, the remaining layers are shared across all languages, and the decoder is

[1]Throughout the paper, we use layer indices starting at 1.



(a) Regular Transformer Encoder Layer.

(b) Language-Specific Transformer Encoder Layer.

Figure 1: Side-by-side comparison of the regular Transformer Encoder Layer (a) and our Language-Specific Transformer Encoder Layer (b). For the Language-Specific Transformer Encoder Layer, an indexing language is provided which routes whole sentences to the appropriate weights (here *Portuguese* ▫).

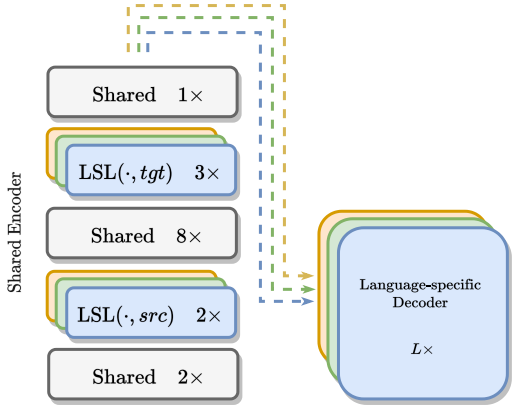


Figure 2: Best-performing separate decoder architecture using LSLs, found using our architecture search method. Layers 3 and 4 are indexed by the source language and layers 13, 14, and 15 are indexed by the target language. The indices start at 1 on the bottom of the encoder.

also target language-specific. For the non-shared layers, we propose using Language-Specific Transformer Layers (LSLs), illustrated in Figure 1b. Quite simply, LSLs are a combination (i.e., a dictionary) of regular Transformer layers (Figure 1a), where the sub-layer used depends on the chosen language. We consider two cases: source-indexed LSLs, and target-indexed LSLs, distinguished by whether we use the source or the target language to select the appropriate sub-layer.

The main contributions of this work are:

1. We propose a way to increase the model capacity per language, without changing the inference speed.

2. We show that the model benefits from having both language-specific and shared components, as well as from having source and target language-specific components.

3. We propose a technique to aid in learning the best architecture, rather than relying purely on manual trial-and-error.

2 Related Work

There exists a vast literature investigating parameter sharing mechanisms for MNMT. Particularly relevant is the shared-encoder, separate-decoder architecture proposed by Dong et al. (2015) which we use as the base for some of our experiments.

Several works analyze which weights should be shared between languages (Sachan and Neubig, 2018; Blackwood et al., 2018; Platanios et al., 2018; Zhu et al., 2020; Wang et al., 2019, 2018). Regardless, most closely related to the presented work are the studies by Zhang et al. (2021) and Purason and Tättar (2022). Zhang et al. (2021) propose adding Conditional Language-Specific Routing (CLSR) layers inside the encoder and decoder Transformer layers. They learn to mix between language-specific and shared weights, and do this on a word by word basis. Our approach does not use learned routing but uses the same components for the whole sentence per language-pair, instead of computing a mixed representation. We also do not add extra parameters to the layer, meaning we have the same inference time complexity as regular Transformer layers. The approach in Purason and Tättar (2022) is similar to ours in the sense that they use language-specific Transformer layers on the encoder side, and also look into sharing weights on a

language-family basis. In contrast to our approach, they focus on source-indexed language-specific layers, while we investigate selecting the layers based on the source or the target language. Besides, we propose a systematic method for deciding which layers to share, and which to be language specific.

Connection to Adapter Layers Adapter Layers (Houlsby et al., 2019; Bapna and Firat, 2019; He et al., 2022) are a lightweight technique to fine-tune a pre-trained encoder model by injecting task-specific sub-modules into the existing architecture. In contrast, LSLs are designed to be trained from scratch, and replace shared by language-specific components, rather than adding new ones, keeping the overall computational costs constant.

Connection to Mixture-of-Experts LSLs enable the introduction of source- and target-specific parameters in the encoder and increase the model capacity, while at the same time keeping the inference cost and effective parameter count for the forward-pass constant (see Figure 1). As such, they are similar in nature to sparsely activated mixture-of-experts layers (MOEs, Shazeer et al., 2017; Roller et al., 2021; Lepikhin et al., 2021) but with the important differences that 1) there is no need for learning a balanced routing module; 2) sub-layer utilization is enforced by design, which tends to be a problem for MOE layers (Dua et al., 2022); and 3) sentences are always routed to the same conditional compute based on the indexing-language, enabling smaller binaries for on-device downloading of model weights as well as consecutive downloads to extend the on-device capabilities to new languages. In fact, Kudugunta et al. (2021) have shown that the final encoder MOE layers also learn target language-specific utilization where a subset of experts is used when translating e.g. X→EN. However, since it is commonly *not* strictly enforced, downloading *all* experts is required, increasing the download size for end users.

3 Methods

In this section we describe our proposed Language-Specific Transformer Layer, as well as a way to select whether to use shared or language-specific weights for each layer.

3.1 Language-Specific Transformer Layer

The idea of LSLs is simple: instead of sharing the same parameters across all languages, have the weights for the layer be language-specific as illustrated in Figure 1. LSLs are composed of one "regular" Transformer encoder layer *per language*. The input is routed to the appropriate sub-layer depending on the source or target language, and at any time only one of the sub-layers is used. Simply replacing all layers in the Transformer with LSLs would significantly increase the number of parameters, and reduce the sharing between languages. For example, if all LSLs are indexed by the source (or target) language it would be identical to a "separate encoder separate decoder" architecture. Instead, we propose a mix of LSLs and regular Transformer layers, which allows the model to learn language-specific and shared weights. See Figure 2 for one such architecture. A sample implementation for FAIRSEQ (Ott et al., 2019) is given in Appendix A.

3.2 Learning the Architecture

Intuitively, we expect the bottom layers of the encoder to require more source language knowledge, while the top ones should already capture target language information as found by Kudugunta et al. (2019). This observation motivates using source-indexed LSLs in the bottom encoder layers, target-indexed LSLs in the top ones, and keeping the remaining layers shared as illustrated in Figure 2. This type of reasoning quickly gets out of hand, as the number of possible architectures is exponential in the numbers of layers. To avoid having to manually select which layers should be shared, and which should be source- or target-indexed LSLs, we propose a Neural Architecture Search (Elsken et al., 2019) inspired approach.

For each layer in the encoder, we learn a shared layer as well as one LSL, which can be source- and target-indexed, and 3 scalar mixing weights:

$$\begin{aligned} \boldsymbol{h}_i = w_i^{shared} &\cdot \text{layer}_i^{shared}(\boldsymbol{h}_{i-1}) &+ \\ w_i^{src} &\cdot \text{LSL}_i(\boldsymbol{h}_{i-1}, src) &+ \\ w_i^{tgt} &\cdot \text{LSL}_i(\boldsymbol{h}_{i-1}, tgt) &, \end{aligned} \tag{1}$$

where $\boldsymbol{h}_{i-1}$ and $\boldsymbol{h}_i$ are the outputs of layers $i-1$ and i, respectively, and $w_i^{shared} + w_i^{src} + w_i^{tgt} = 1$. $\text{LSL}_i(\boldsymbol{h}_{i-1}, src)$ means we select the LSL weights by the source language, while $\text{LSL}_i(\boldsymbol{h}_{i-1}, tgt)$ corresponds to using the target weights.

As there is no constraint on the mixing weights, other than that they sum to 1 and are non-negative[2], the model is incentivized to use all the sub-layers,

[2]We implement this constraint by applying the $\mathrm{softmax}$ function to the 3 scalar parameters.

resulting in a huge increase in the number of parameters. If we have L different languages, then each layer will have as many parameters as $L + 1$ "regular" Transformer layers.[3] The amount of computation increases by a factor of 3, as we compute three intermediate representations: a shared one, one using the source language sub-layer, and another using the target language sub-layer, which we then mix according to Equation (1).

To keep the inference time unaffected and the model size reasonable, only one of the components should be used, i.e., the mixing weights should be sparse. In this work, we propose a simple but effective approach: for each layer, we select the component with the largest converged weight. For example, if the largest weight for layer i is w_i^{tgt}, then layer i will be a target-indexed LSL. After selecting the architecture, we train it from scratch.

3.3 Dense Pre-training

Inspired by Dua et al. (2022), we found that initializing all encoder weights (both shared and LSLs) from a pre-trained architecture consisting only of "regular" Transformer layers helped achieve better performance. In our experiments, we copy the pre-trained weights from the respective layers to the language-specific modules for initialization. The pre-trained weights come from our baseline architectures, shared-encoder models with only "regular" Transformer Layers. We use the separate decoder baseline's weights for the separate decoder models (e.g., LSL-NAS), and the shared decoder baseline's weights for the shared decoder models (e.g., LSL-NAS-SD). This procedure has multiple advantages: 1) It maximizes cross-lingual transfer by training a general representation across languages first and minimizes language interference during fine-tuning; 2) It mitigates under-trained language-specific components for low-resource languages as they usually see significantly less data and the naïve approach of training with higher sampling temperatures typically degrades performance on high resource languages (Arivazhagan et al., 2019; Wang et al., 2020); and 3) it improves convergence speed for architectures with LSLs.

4 Results

In the following, we will describe our experiments and discussion regarding the effectiveness of LSLs.

[3]Plus, of course, the mixing weights, but they amount to only 3 extra parameters per layer.

4.1 Experimental Setup

Data In our experiments, we focus on the following 10 languages: German (DE), English (EN), Spanish (ES), French (FR), Italian (IT), Japanese (JA), Korean (KO), Portuguese (PT), Swahili (SW), and Chinese (ZH). We collect data for these languages from the WMT21 news translation task sources (composed of Europarl v10, ParaCrawl v7.1, ParaCrawl v8, Common Crawl, News Commentary v16, Wiki Titles v3, UN Parallel Corpus V1.0, Tilde Rapid, WikiMatrix, Back-translated news, Japanese-English Subtitle Corpus, The Kyoto Free Translation Task Corpus, and TED Talks) as well as Opus-100 (Zhang et al., 2020), Tatoeba (Tiedemann, 2012), and CCMatrix (Schwenk et al., 2021). We deduplicate the data, and preprocess it using the M2M-100 (Fan et al., 2021) scripts.[4] The final dataset sizes can be seen in Appendix B.

Since CCMatrix is a large yet low quality data source, we found it helpful to downsample it relative to the other sources using temperature sampling. For more details, see Appendix B.

Evaluation For evaluation, we use the dev and devtest splits of the Flores-101 dataset (Goyal et al., 2022) as our validation and test sets, respectively. Except when stated otherwise, the reported numbers are on the test set. We report both CHRF (Popović, 2015) and SPBLEU (Goyal et al., 2022), a SENTENCEPIECE-based BLEU computed using the Flores-101 tokenizer, with `sacreBLEU`[5] version 2.3.1. The evaluation signatures are `nrefs:1 | case:mixed | eff:no | tok:flores101 | smooth:exp` for SPBLEU, and `nrefs:1 | case:mixed | eff:yes | nc:6 | nw:0 | space:no` for CHRF. All our results are from a single training run of each architecture, and we perform statistical significance tests using paired bootstrap resampling (Koehn, 2004). We run the significance tests for CHRF for all language directions, using a significance level of 5%. We also provide COMET scores (Rei et al., 2020)[6] for selected models in Appendix G.

Tokenization We use SENTENCEPIECE (Kudo and Richardson, 2018), with a vocabulary size of 250k, and a character coverage of 0.9995. We balance the data for SENTENCEPIECE training by ran-

[4]https://github.com/facebookresearch/fairseq/tree/main/examples/m2m_100

[5]https://github.com/mjpost/sacrebleu

[6]Obtained with `wmt20-comet-da` from version `1.1.2`.

domly sampling 1.5M sentences per language.

Tagging We found it helpful to make the model aware of the corpus by training with corpus labels. Similarly to NLLB Team et al. (2022), we add a tag (e.g. <HQ> or <LQ>) to the beginning of the source sentence, so that the model can learn to distinguish between higher quality (WMT21, Opus-100, and Tatoeba) and lower quality examples (CCMatrix). During inference, we always use the high quality (<HQ>) tag. Additionally, we append source and target language tags to the end of the sentence.

Architecture In our experiments, we use a deep encoder, shallow decoder architecture (Kasai et al., 2021) with 16 encoder layers and 3 decoder layers. We share token embeddings between the encoder, decoder, and output layer (Press and Wolf, 2017). In our experiments we consider two kinds of models: those with target language-specific decoders, following Dong et al. (2015), on which we conduct most of our experiments, and those with a shared decoder. The encoder is always shared, with the exception of the LSLs. In the baseline models, the encoder consists only of "regular" Transformer Layers, and so it is fully shared. In this work, we only consider adding LSLs to the encoder. In preliminary experiments with LSLs in the decoder, our selection criteria picked target-specific LSLs for all decoder layers, effectively choosing a separate decoder architecture. We tried different placements of the layers in the decoder, but did not achieve any improvements. We leave a deeper analysis to future work.

Hyperparameters All experiments are implemented using FAIRSEQ (Ott et al., 2019). We use ADAM (Kingma and Ba, 2015) for optimization, due to its robustness (Schmidt et al., 2021) and popularity, with a learning rate of 0.0004. We train for 150k steps, by which point our models had converged, with 4000 warm-up steps, and an inverse square root learning rate scheduler (Vaswani et al., 2017). Due to the abundance of data, adding regularization in the form of dropout or weight decay did not help in our initial experiments, so we do not use any regularization in the remaining experiments. The layer and embedding sizes are 512, the hidden size of the feed-forward layers is 2048, and we use 8 attention heads. All models are trained using `fp16` (Ott et al., 2018).

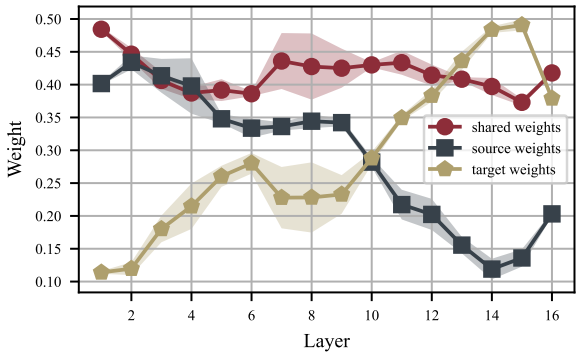


Figure 3: Converged mixing weights across layers, averaged over 3 runs. The model shows a preference for source LSLs near the bottom of the encoder, target LSLs near the top, and shared layers in between. Shaded regions show the uncertainty.

4.2 Architecture Search

As described in Section 3.2, we train a separate-decoder model where all encoder layers are a mix of shared, source, and target weights. This architecture used a total of 804 million (M) parameters, and achieved a score of 46.6 CHRF points (27.4 SPBLEU), averaged over all language pairs. We plot the mixing coefficients of the model in Figure 3, averaged over 3 runs.

We can see clear trends here: the model gives a higher weight to the source-specific sub-layers near the bottom of the encoder, while the target-specific sub-layers get a higher weight near the top. This is in line with previous studies as lower encoder layers usually capture low-level information about the source (Tenney et al., 2019), while the top encoder layers are known to already capture target language information (Kudugunta et al., 2019). Interestingly, the mixing coefficients for the shared weights are relatively stable across layers, making them dominant for the middle layers of the model.

Taking the $\arg\max$ of the mixing coefficients, we select the architecture in Figure 2, where layers 3 and 4 are source-indexed LSLs[7], layers 13, 14, 15 are target-indexed LSLs, and the remaining layers are "regular" Transformer encoder layers (Figure 1a). From here onward, we will refer to this architecture as LSL-NAS. We use the architecture selection method only to select the architecture, and the selected architecture is trained from scratch (**not pruned**) in the upcoming experiments. To

[7]For these layers there is some uncertainty in the source weights, but they are the largest weights by a small margin. Performance is improved by selecting source layers, as can be attested by comparing to LSL (SRC=$\emptyset$ & TGT=$\{13, 14, 15\}$).

Model	CHRF	SPBLEU	$\lvert\boldsymbol{\theta}\rvert$	$\lvert\boldsymbol{\theta}_{\text{eff}}\rvert$
Separate Decoder Baseline	45.5	26.0	299M	186M
+ hidden dim 640	45.9	26.6	399M	240M
+ hidden dim 704	46.2	26.9	453M	268M
+ hidden dim 768	46.5	27.3	509M	296M
Language Adapters ENC 128	45.8	26.4	321M	207M
Language Adapters ENC 256	45.7	26.3	342M	228M
Language Adapters ENC 512	45.6	26.3	384M	270M
Language Adapters ENC (SRC+TGT) 128	45.7	26.3	321M	207M
Language Adapters ENC (SRC+TGT) 256	46.1	26.7	342M	228M
Language Adapters ENC (SRC+TGT) 512	46.0	26.7	384M	270M
LSL-NAS	46.4	27.2	441M	186M
+ *Dense Pre-training*	**46.8**	**27.5**	441M	186M
LSL (SRC=$\{1,2\}$ & TGT=$\{15,16\}$)	46.3	27.0	413M	186M
LSL (SRC=$\{1,2,3\}$ & TGT=$\{14,15,16\}$)	46.2	27.0	470M	186M
LSL (SRC=$\emptyset$ & TGT=$\{13,14,15\}$)	45.8	26.5	385M	186M
LSL (SRC=$\{3,4\}$ & TGT=$\emptyset$)	46.1	26.7	356M	186M
LSL (SRC=$\{13,14,15\}$ & TGT=$\{3,4\}$)	45.2	25.7	441M	186M

Table 1: Comparison of different separate decoder models. Although the total number of parameters in the model $|\boldsymbol{\theta}|$ changes by adding more LSLs, the effective parameter count $|\boldsymbol{\theta}_{\text{eff}}|$ stays consistent for all translation directions due to sparse language-dependent activations.

simplify the text, we will also use the notation LSL (SRC=$\{1,2\}$ & TGT=$\{15,16\}$) to refer to an architecture with source-indexed LSLs in layers 1 and 2, and target-indexed LSLs in layers 15 and 16.

4.3 Learned Architecture Comparison

In Table 1, we compare our baseline separate decoder architecture (with a fully shared 16 layer encoder) with the learned architecture from the architecture search (LSL-NAS), and additional variants. We share CHRF and SPBLEU scores averaged over all language pairs, as well as the number of total ($|\boldsymbol{\theta}|$) and effective ($|\boldsymbol{\theta}_{\text{eff}}|$) parameters used during inference for each architecture. For the baseline models, $|\boldsymbol{\theta}|$ and $|\boldsymbol{\theta}_{\text{eff}}|$ differ due to the separate decoders. For an accurate comparison of CPU and GPU speed, see Appendix H.

Our learned architecture (LSL-NAS in Table 1) achieves a 0.9 CHRF (1.2 SPBLEU) improvement over the baseline, which we can be further increased to 1.3 with dense pre-training, reaching a total of 46.8 CHRF (27.5 SPBLEU). These improvements are statistically significant ($p < 0.05$) for all but 6 of the 90 translation directions. In Table 2, we summarize the averaged results for translating to and from each language, i.e. X→ DE is the average CHRF score for translating into German from all other languages. For the full results (per language pair) on the validation and test sets, see Appendix C. Our approach gives substantial gains for both high resource languages, such as English and German, which improve by more than 1 CHRF point, as well as lower resource, such as Korean, with close to 2 CHRF points improvement for both directions, or Swahili, which improves by over 1.5 CHRF points in both directions. Although the effective number of parameters is the same for this architecture and our baseline (186M), it can be argued that this comparison is unfair, since our model is bigger. To alleviate this concern, and to show that the gains we achieve are not just due to the higher parameter count, but rather, the better way we allocate the extra parameters, we trained three bigger baselines: with hidden sizes of 640, 704, and 768. As expected, these models also show an improvement over the original baseline, but even the biggest model, with a total of 509M parameters (15% more than ours) and a higher inference cost than our method, is not able to match our performance (only 46.5 CHRF and 27.3 SPBLEU).

Adapters Following Philip et al. (2020), we insert one adapter block after each Transformer layer. In our experiments, inserting adapters into a pre-trained model either provided no improvement over training from scratch or suffered from numerical instabilities, even after tuning the initialization gain (Houlsby et al., 2019). For this reason we report numbers for models trained from scratch, similar to Baziotis et al. (2022). Since our models have separate decoders, we inserted adapters only on the encoder. For completeness, results using adapters on the decoder are reported in Appendix D.

We consider two kinds of adapters: source language adapters (Language Adapters ENC), following Philip et al. (2020), or source language adapters in the bottom half of the encoder and target language language adapters in the top half (Language Adapters ENC (SRC+TGT)). We show the results for different bottleneck dimensions (128, 256, and 512) in Table 1. Our proposal of using source and target adapters on the encoder outperforms using only source adapters (for the same model size). The best performing model, Language Adapters ENC (SRC+TGT), achieves a score of 46.1 CHRF points, 0.3 (0.7) points lower than our model without (with) dense pre-training.These improvements are statistically significant ($p < 0.05$) for 38 (62) of the 90 translation directions. Results for language-pair adapters (Bapna and Firat, 2019) are shown in Appendix D, but they lag behind language adapters.

Model	DE	EN	ES	FR	IT	JA	KO	PT	SW	ZH
Translating *into* the language (X → __)										
Separate Decoder Baseline	52.7	60.4	49.1	57.0	50.6	29.0	25.2	54.9	47.9	28.7
Language Adapters ENC (SRC+TGT) 256	53.2	60.9	49.5	57.7	50.9	29.9	25.0	55.4	49.1	29.1
LSL-NAS	**53.9**	**61.5**	**49.9**	**58.1**	**51.4**	**31.0**	**27.0**	**55.6**	**49.4**	**29.9**
Translating *from* the language (__ → X)										
Separate Decoder Baseline	47.7	52.7	44.9	48.1	46.4	42.1	40.0	49.4	40.1	44.0
Language Adapters ENC (SRC+TGT) 256	48.1	53.0	45.4	48.5	46.7	42.8	41.2	49.9	40.8	44.5
LSL-NAS	**48.7**	**53.6**	**45.8**	**48.9**	**47.3**	**43.5**	**42.1**	**50.4**	**42.1**	**45.3**

Table 2: Comparison of LSL-NAS with pre-training, the separate decoder baseline model and the best separate decoder adapter model, per source and target language. Our approach gives substantial average CHRF improvements over the baseline (adapter) model, which are statistically significant for 84 (62) of the 90 translation directions.

Model	CHRF	SPBLEU
LSL-NAS	**46.4**	**27.2**
LSL (SRC={3, 4} & TGT={14, 15, 16})	46.4	27.0
LSL (SRC={2, 3} & TGT={13, 14, 15})	46.4	27.1
LSL (SRC={1, 2} & TGT={13, 14, 15})	46.2	26.9
LSL (SRC={1, 2} & TGT={14, 15, 16})	46.2	26.9

Table 3: Influence of the shared layers on the bottom and the top of the encoder. Our learned architecture, LSL-NAS, is LSL (SRC={3, 4} & TGT={13, 14, 15}).

Model	CHRF	SPBLEU	$\lvert\boldsymbol{\theta}\rvert$	$\lvert\boldsymbol{\theta}_{\text{eff}}\rvert$
LSL-NAS	**46.4**	**27.2**	441M	186M
LS-FFN	46.3	26.8	394M	186M
LS-ATTENTION	45.9	26.5	347M	186M

Table 4: Effect of each Transformer layer component. LSL uses full language-specific layers. LS-FFN shares the attention, but keeps the feed-forwards language specific, while LS-ATTENTION does the opposite. All experiments are based on the LSL-NAS architecture and differ only in their language-specific components.

Importance of Bottom and Top Shared Layers LSL-NAS uses two shared layers on the bottom and one shared layer on the top of the encoder. In Table 3, we analyze the effect of removing these layers, i.e., moving the LSLs up or down. When comparing SPBLEU there is a small drop when removing either the top shared layer (row "LSL (SRC={3, 4} & TGT={14, 15, 16})") or the bottom-most shared layer (row "LSL (SRC={2, 3} & TGT={13, 14, 15})"), but the difference is negligible when comparing CHRF. In fact the difference is only statistically significant for 15 of the 90 translation directions. When removing the bottom shared layers (row "LSL (SRC={1, 2} & TGT={13, 14, 15})") or all the shared layers (row "LSL (SRC={1, 2} & TGT={14, 15, 16})"), there is a bigger difference, but it is only statistically significant for less than 1/3 of the translation directions, mostly low resource pairs including either Swahili or Korean. For an analysis regarding the number of LSLs, please refer to Appendix E.

Alternative Configurations Additionally, we look at different configurations of LSLs. In particular, we compare using only source-specific layers LSL (SRC={3, 4} & TGT=∅) or target-specific layers LSL (SRC=∅ & TGT={13, 14, 15}) in Table 1. In both cases, the configuration is worse than LSL-NAS, thus showing the importance of having both source and target-specific layers. For completeness, row LSL (SRC={13, 14, 15} & TGT={3, 4}) shows the opposite of our configuration (i.e., swapping the source and target layers), with considerably degraded performance, showing that the position of source-specific and target-specific languages is very important. In particular, it shows that forcing the model to learn source-specific representations at higher encoder layers and target language representations on the lower layers hinders learning.

Layer Component Ablation We analyze the effect of using full language-specific layers (LSL), with having only language-specific feed-forward (LS-FFN), or only language-specific attention (LS-ATTENTION) on the LSL-NAS architecture in Table 4. We observe a small degradation of 0.1 CHRF (0.4 SPBLEU) when switching from LSL to LS-FFN, which is statistically significant for 35/90 translation directions, and a degradation of 0.5 CHRF (0.7 SPBLEU) when switching to LS-ATTENTION, which is significant for 49 directions. These results imply that both the language-specific feed-forward and attention are important, with the biggest contribution coming from the feed-forward part, where most of the parameters are located.

4.4 Shared Decoder

So far we have focused on the separate-decoder architecture. In this section, we turn to a shared-decoder setup (see Table 5). As in Section 4.2, we ran an architecture search experiment and selected the following architecture: LSL (SRC=$\{4\}$ & TGT=$\{12, 13, 14, 15, 16\}$), or LSL-NAS-SD for short. The mixing weights follow a trend similar to Figure 3. With the shared decoder, we benefit from placing more target-specific layers at the top of the encoder. Our intuition is that these layers compensate the lack of a separate decoder.

As in Section 4.3, we compare against shared decoder baseline models (i.e., without LSLs) of increasing sizes, as well as models with Adapter Blocks. For the latter, we insert one block after each Transformer layer, both on the encoder and the decoder. Following Philip et al. (2020), we insert source adapters on the encoder, and target adapters on the decoder. As expected, shared-decoder models perform worse than their separate-decoder models, which have a higher parameter count. Despite this, our proposed architecture, LSL-NAS-SD, outperforms the remaining models by a wide margin, and is even better than the separate-decoder baseline (26.0 SPBLEU). The improvements of our LSL-NAS-SD model with pre-training over the shared decoder baseline are statistically significant for $86/90$ translation directions. The improvements over the best adapter model (bottleneck size 512) are significant for $76/90$ directions.

We also show the performance for LSL (SRC=$\{4\}$ & TGT=$\{13-16\}$), an architecture similar to LSL-NAS-SD, but with one less target-specific LSL. This architecture performs worse than our selection, but has fewer parameters, which might make it a preferable candidate for deployment. This highlights a limitation of our selection approach: it does not take model complexity (i.e., model size) into account. We tried adding a prior on the mixing weights to make LSLs more costly than shared layers, but obtained mixed results, and we leave further investigation to future work.

4.5 Zero-shot Translation

In the previous experiments, we used training data for all language directions. We now consider a different scenario: we limit our training data to English directions (i.e., X-EN and EN-X) and languages in the same language group[8]. We then evaluate our models on zero shot performance for the directions between groups.

Model	CHRF	SPBLEU	$\lvert\boldsymbol{\theta}\rvert$	$\lvert\boldsymbol{\theta}_{\text{eff}}\rvert$
Separate Decoder Baseline	45.5	26.0	299M	186M
LSL-NAS (separate decoder)	46.4	27.2	441M	186M
Shared Decoder Baseline	44.7	24.9	186M	186M
+ hidden dim 640	45.1	25.5	240M	240M
+ hidden dim 704	45.8	26.2	268M	268M
+ hidden dim 768	45.8	26.3	296M	296M
Shared Decoder Adapters 128	44.6	24.8	211M	189M
Shared Decoder Adapters 256	44.9	25.0	236M	191M
Shared Decoder Adapters 512	45.3	25.6	286M	196M
Shared Decoder Adapters 640	45.3	25.5	311M	199M
LSL-NAS-SD	46.3	26.7	356M	186M
+ *Dense Pre-training*	**46.6**	**27.1**	356M	186M
LSL (SRC=$\{4\}$ & TGT=$\{13-16\}$)	46.1	26.5	328M	186M

Table 5: Results on the shared-decoder architecture.

Direction	Baseline	Adapters	LSL-NAS-SD
Overall Average	39.9	**41.8**	41.4
Overall Average (w/o *→SW)	40.8	42.5	**44.7**
Zero-shot Average	29.6	**32.4**	31.9
Zero-shot Average (w/o *→SW)	29.3	32.0	**36.2**
EUR → CJK	23.8	18.5	**27.6**
EUR → SW	34.2	**37.3**	11.7
CJK → EUR	41.4	45.1	**45.5**
CJK → SW	24.8	**28.9**	11.7
SW → EUR	23.5	44.9	**46.4**
SW → CJK	6.70	12.3	**18.7**

Table 6: Zero-Shot comparison of the shared decoder models. The global average includes non zero-shot directions. The remaining scores are all zero-shot.

In our initial experiments, separate decoder models performed poorly on zero-shot directions, so we focused our evaluation on shared decoder models. Table 6 shows the zero-shot results for 3 architectures: the shared decoder baseline, the best performing (shared decoder) adapter model (Shared Decoder Adapters 512), and LSL-NAS-SD. Our approach gives improvements for most zero-shot directions, except when translating *into* SW. Translating *from* SW works well, though. Our intuition is that this degradation is caused by the SW target-specific LSLs being overfitted to EN, and thus failing to transfer to other languages. In LSL-NAS-SD, the top 5 encoder layers are target LSLs, and in the zero-shot scenario, the SW layers are only trained for EN-SW, which is relatively small. Indeed, if we exclude the * →SW pairs, both the overall and the zero-shot average scores increase.

[8]We consider 3 groups: European, CJK, and Swahili. We use data where both the source and target languages are in the same group.

5 Conclusion

In this work, we studied how to increase the capacity of MNMT models using LSLs. We showed that LSLs are effective at increasing the model capacity per language, while keeping the computation requirements constant. We proposed a method for selecting the placement of LSLs, and showed the importance of having shared as well as source and target language-specific parameters on the encoder.

Limitations

In this work, we focused our exploration of LSLs on the encoder. Although we ran some initial explorations on the decoder side, further investigation is needed. Another venue for research is how LSLs affect language expansion. Since our approach tries to limit the language-specific weights to just a few layers, *in theory*, it should be possible to add new languages by only expanding and training the LSLs. However, blindly doing so might not work well and the interactions between languages from different families needs further studying. Lastly, it is unclear whether our $\arg\max$ approach to selecting where to place LSLs is optimal, how dataset dependent it is, and if there exist alternative approaches that can lead to better results. The fact that it does not take model complexity (i.e., model size) into account can be a disadvantage in practice.

Ethics Statement

Our work uses existing datasets, so it inherits some of the risks associated with them, namely gender bias (Cho et al., 2019), or privacy leakage (Carlini et al., 2021), and mitigation strategies such as Vanmassenhove et al. (2018) may be necessary. However, replacing bilingual translation systems with multilingual systems should help reduce gender bias caused by pivoting through English. Another consideration is the energy consumption for model training, which results in green-house emissions (Strubell et al., 2019). Our proposed architectures result in smaller (and faster to train) models, than similarly-performing baselines, increasing the efficiency of translation systems.

Acknowledgements

We would like to thank Sarthak Garg, Luke Carlson, António V. Lopes, and Matthias Sperber for their comments and suggestions, which significantly improved the final work.

References

Roee Aharoni, Melvin Johnson, and Orhan Firat. 2019. Massively multilingual neural machine translation. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 3874–3884, Minneapolis, Minnesota. Association for Computational Linguistics.

Naveen Arivazhagan, Ankur Bapna, Orhan Firat, Dmitry Lepikhin, Melvin Johnson, Maxim Krikun, Mia Xu Chen, Yuan Cao, George F. Foster, Colin Cherry, Wolfgang Macherey, Zhifeng Chen, and Yonghui Wu. 2019. Massively multilingual neural machine translation in the wild: Findings and challenges. *CoRR*, abs/1907.05019.

Ankur Bapna and Orhan Firat. 2019. Simple, scalable adaptation for neural machine translation. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 1538–1548, Hong Kong, China. Association for Computational Linguistics.

Christos Baziotis, Mikel Artetxe, James Cross, and Shruti Bhosale. 2022. Multilingual Machine Translation with Hyper-Adapters. *ArXiv*, abs/2205.10835.

Graeme Blackwood, Miguel Ballesteros, and Todd Ward. 2018. Multilingual neural machine translation with task-specific attention. In *Proceedings of the 27th International Conference on Computational Linguistics*, pages 3112–3122, Santa Fe, New Mexico, USA. Association for Computational Linguistics.

Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. 2021. Extracting training data from large language models. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2633–2650. USENIX Association.

Won Ik Cho, Ji Won Kim, Seok Min Kim, and Nam Soo Kim. 2019. On measuring gender bias in translation of gender-neutral pronouns. In *Proceedings of the First Workshop on Gender Bias in Natural Language Processing*, pages 173–181, Florence, Italy. Association for Computational Linguistics.

Daxiang Dong, Hua Wu, Wei He, Dianhai Yu, and Haifeng Wang. 2015. Multi-task learning for multiple language translation. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 1723–1732, Beijing, China. Association for Computational Linguistics.

Dheeru Dua, Shruti Bhosale, Vedanuj Goswami, James Cross, Mike Lewis, and Angela Fan. 2022. Tricks for training sparse translation models. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3340–3345, Seattle, United States. Association for Computational Linguistics.

Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. 2019. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21.

Angela Fan, Shruti Bhosale, Holger Schwenk, Zhiyi Ma, Ahmed El-Kishky, Siddharth Goyal, Mandeep Baines, Onur Celebi, Guillaume Wenzek, Vishrav Chaudhary, Naman Goyal, Tom Birch, Vitaliy Liptchinsky, Sergey Edunov, Michael Auli, and Armand Joulin. 2021. Beyond english-centric multilingual machine translation. *Journal of Machine Learning Research*, 22(107):1–48.

Markus Freitag and Orhan Firat. 2020. Complete multilingual neural machine translation. In *Proceedings of the Fifth Conference on Machine Translation*, pages 550–560, Online. Association for Computational Linguistics.

Naman Goyal, Cynthia Gao, Vishrav Chaudhary, Peng-Jen Chen, Guillaume Wenzek, Da Ju, Sanjana Krishnan, Marc'Aurelio Ranzato, Francisco Guzmán, and Angela Fan. 2022. The Flores-101 evaluation benchmark for low-resource and multilingual machine translation. *Transactions of the Association for Computational Linguistics*, 10:522–538.

Junxian He, Chunting Zhou, Xuezhe Ma, Taylor Berg-Kirkpatrick, and Graham Neubig. 2022. Towards a unified view of parameter-efficient transfer learning. In *10th International Conference on Learning Representations, ICLR*, virtual.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning, ICML*, volume 97 of *Proceedings of Machine Learning Research*, pages 2790–2799, Long Beach, CA, USA. PMLR.

Melvin Johnson, Mike Schuster, Quoc V. Le, Maxim Krikun, Yonghui Wu, Zhifeng Chen, Nikhil Thorat, Fernanda Viégas, Martin Wattenberg, Greg Corrado, Macduff Hughes, and Jeffrey Dean. 2017. Google's multilingual neural machine translation system: Enabling zero-shot translation. *Transactions of the Association for Computational Linguistics*, 5:339–351.

Jungo Kasai, Nikolaos Pappas, Hao Peng, James Cross, and Noah A. Smith. 2021. Deep encoder, shallow decoder: Reevaluating non-autoregressive machine translation. In *9th International Conference on Learning Representations, ICLR*, virtual. OpenReview.net.

Diederik P. Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*.

Philipp Koehn. 2004. Statistical significance tests for machine translation evaluation. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pages 388–395, Barcelona, Spain. Association for Computational Linguistics.

Taku Kudo and John Richardson. 2018. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, Brussels, Belgium. Association for Computational Linguistics.

Sneha Kudugunta, Ankur Bapna, Isaac Caswell, and Orhan Firat. 2019. Investigating multilingual NMT representations at scale. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 1565–1575, Hong Kong, China. Association for Computational Linguistics.

Sneha Kudugunta, Yanping Huang, Ankur Bapna, Maxim Krikun, Dmitry Lepikhin, Minh-Thang Luong, and Orhan Firat. 2021. Beyond distillation: Task-level mixture-of-experts for efficient inference. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 3577–3599, Punta Cana, Dominican Republic. Association for Computational Linguistics.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. Gshard: Scaling giant models with conditional computation and automatic sharding. In *9th International Conference on Learning Representations, ICLR*, virtual. OpenReview.net.

NLLB Team, Marta R. Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, Anna Sun, Skyler Wang, Guillaume Wenzek, Al Youngblood, Bapi Akula, Loïc Barrault, Gabriel Mejia Gonzalez, Prangthip Hansanti, John Hoffman, Semarley Jarrett, Kaushik Ram Sadagopan, Dirk Rowe, Shannon Spruit, Chau Tran, Pierre Andrews, Necip Fazil Ayan, Shruti Bhosale, Sergey Edunov, Angela Fan, Cynthia Gao, Vedanuj Goswami, Francisco Guzmán, Philipp Koehn, Alexandre Mourachko, Christophe Ropers, Safiyyah Saleem, Holger Schwenk, and Jeff Wang. 2022. No language left behind: Scaling human-centered machine translation. *CoRR*, abs/2207.04672.

Myle Ott, Sergey Edunov, Alexei Baevski, Angela Fan, Sam Gross, Nathan Ng, David Grangier, and Michael Auli. 2019. fairseq: A fast, extensible toolkit for sequence modeling. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (Demonstrations)*, pages 48–53, Minneapolis, Minnesota. Association for Computational Linguistics.

Myle Ott, Sergey Edunov, David Grangier, and Michael Auli. 2018. Scaling neural machine translation. In *Proceedings of the Third Conference on Machine Translation: Research Papers*, pages 1–9, Brussels, Belgium. Association for Computational Linguistics.

Jerin Philip, Alexandre Berard, Matthias Gallé, and Laurent Besacier. 2020. Monolingual adapters for zero-shot neural machine translation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4465–4470, Online. Association for Computational Linguistics.

Emmanouil Antonios Platanios, Mrinmaya Sachan, Graham Neubig, and Tom Mitchell. 2018. Contextual parameter generation for universal neural machine translation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 425–435, Brussels, Belgium. Association for Computational Linguistics.

Maja Popović. 2015. chrF: character n-gram F-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation*, pages 392–395, Lisbon, Portugal. Association for Computational Linguistics.

Ofir Press and Lior Wolf. 2017. Using the output embedding to improve language models. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, pages 157–163, Valencia, Spain. Association for Computational Linguistics.

Taido Purason and Andre Tättar. 2022. Multilingual neural machine translation with the right amount of sharing. In *Proceedings of the 23rd Annual Conference of the European Association for Machine Translation*, pages 91–100, Ghent, Belgium. European Association for Machine Translation.

Ricardo Rei, Craig Stewart, Ana C Farinha, and Alon Lavie. 2020. COMET: A neural framework for MT evaluation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2685–2702, Online. Association for Computational Linguistics.

Stephen Roller, Sainbayar Sukhbaatar, Arthur Szlam, and Jason Weston. 2021. Hash layers for large sparse models. In *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems*, pages 17555–17566, virtual.

Devendra Sachan and Graham Neubig. 2018. Parameter sharing methods for multilingual self-attentional translation models. In *Proceedings of the Third Conference on Machine Translation: Research Papers*, pages 261–271, Brussels, Belgium. Association for Computational Linguistics.

Robin M. Schmidt, Frank Schneider, and Philipp Hennig. 2021. Descending through a crowded valley - benchmarking deep learning optimizers. In *Proceedings of the 38th International Conference on Machine Learning, ICML*, volume 139 of *Proceedings of Machine Learning Research*, pages 9367–9376, virtual. PMLR.

Holger Schwenk, Guillaume Wenzek, Sergey Edunov, Edouard Grave, Armand Joulin, and Angela Fan. 2021. CCMatrix: Mining billions of high-quality parallel sentences on the web. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 6490–6500, Online. Association for Computational Linguistics.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *5th International Conference on Learning Representations, ICLR*, Toulon, France. OpenReview.net.

Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2019. Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3645–3650, Florence, Italy. Association for Computational Linguistics.

Ian Tenney, Dipanjan Das, and Ellie Pavlick. 2019. BERT rediscovers the classical NLP pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4593–4601, Florence, Italy. Association for Computational Linguistics.

Jörg Tiedemann. 2012. Parallel data, tools and interfaces in opus. In *Proceedings of the Eight International Conference on Language Resources and Evaluation (LREC'12)*, Istanbul, Turkey. European Language Resources Association (ELRA).

Chau Tran, Shruti Bhosale, James Cross, Philipp Koehn, Sergey Edunov, and Angela Fan. 2021. Facebook AI's WMT21 news translation task submission. In *Proceedings of the Sixth Conference on Machine Translation*, pages 205–215, Online. Association for Computational Linguistics.

Eva Vanmassenhove, Christian Hardmeier, and Andy Way. 2018. Getting gender right in neural machine translation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3003–3008, Brussels, Belgium. Association for Computational Linguistics.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.

Xinyi Wang, Yulia Tsvetkov, and Graham Neubig. 2020. Balancing training for multilingual neural machine translation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8526–8537, Online. Association for Computational Linguistics.

Yining Wang, Jiajun Zhang, Feifei Zhai, Jingfang Xu, and Chengqing Zong. 2018. Three strategies to improve one-to-many multilingual translation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2955–2960, Brussels, Belgium. Association for Computational Linguistics.

Yining Wang, Long Zhou, Jiajun Zhang, Feifei Zhai, Jingfang Xu, and Chengqing Zong. 2019. A compact and language-sensitive multilingual translation method. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 1213–1223, Florence, Italy. Association for Computational Linguistics.

Jian Yang, Shuming Ma, Haoyang Huang, Dongdong Zhang, Li Dong, Shaohan Huang, Alexandre Muzio, Saksham Singhal, Hany Hassan, Xia Song, and Furu Wei. 2021. Multilingual machine translation systems from Microsoft for WMT21 shared task. In *Proceedings of the Sixth Conference on Machine Translation*, pages 446–455, Online. Association for Computational Linguistics.

Biao Zhang, Ankur Bapna, Rico Sennrich, and Orhan Firat. 2021. Share or not? learning to schedule language-specific capacity for multilingual translation. In *9th International Conference on Learning Representations, ICLR*, virtual. OpenReview.net.

Biao Zhang, Philip Williams, Ivan Titov, and Rico Sennrich. 2020. Improving massively multilingual neural machine translation and zero-shot translation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1628–1639, Online. Association for Computational Linguistics.

Changfeng Zhu, Heng Yu, Shanbo Cheng, and Weihua Luo. 2020. Language-aware interlingua for multilingual neural machine translation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1650–1655, Online. Association for Computational Linguistics.

A LSLs in FAIRSEQ

Listing 1 shows our implementation of LSL in FAIRSEQ. The implementation is straightforward, and consists of a dictionary that selects the appropriate language depending on the `lang_pair` attribute, which FAIRSEQ dynamically sets, and is guaranteed to match that of the input.

B Dataset sizes

For most language pairs, CCMatrix is the largest data source, and it is also the lowest quality one. To compensate for this quality imbalance, we apply temperature sampling (Arivazhagan et al., 2019) to balance the different sources, using a temperature of 5, which worked well in our experiments. In our initial experiments, we considered two approaches to apply this temperature re-sampling: either upsampling the higher quality sources (WMT21, Opus-100, and Tatoeba), or downsampling CCMatrix. The results between these two approaches were similar, and since the downsampling runs were faster and more stable, we used the downsampling for all our experiments. To avoid discarding too much data, we capped the maximum downsampling to a factor of 10.

Table 7 shows the number of sentence pairs for each language direction, after de-duplication, cleaning, and downsampling CCMatrix.

C Full results

Table 8 shows the CHRF scores on the Flores-101 test set for all language directions, of both our shared-encoder, separate-decoder baseline model and our proposed LSL-NAS architecture with pre-training. Statistically non-significant results ($p \geq 0.05$) are marked with † (in total 6 of the 90 language pairs). The results on the validation set can be found in Table 9.

D Results on different configurations

In Table 10 we show the results of further experiments with Adapter Blocks. Besides encoder source language adapters (Language Adapters ENC) and source adapters in the bottom half of the encoder and target adapters in the top half (Language Adapters ENC (SRC+TGT), we include source adapters on the encoder and target adapters on the decoder (Language Adapters ENC+DEC, like Philip et al. (2020), and language-pair adapters on the encoder (Bapna and Firat, 2019) (Language-Pair Adapters ENC). Our proposed architecture, LSL-NAS, outperforms all other techniques while introducing no extra computation at inference time (i.e., it keeps $|\boldsymbol{\theta}_{\text{eff}}|$ constant).

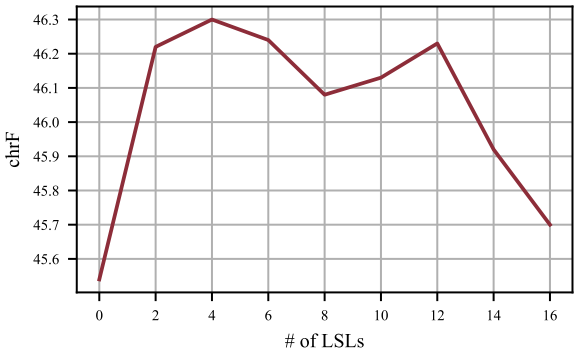


Figure 4: Average CHRF scores over all languages versus the number of LSLs. For each data point, half of the LSLs are on the bottom of the encoder, and the other half are on the top, e.g. for 4 LSLs, the bottom 2 layers are source-indexed, the top 2 are target-indexed, and the remaining layers are shared.

E Number of LSLs

We look at the effect of changing the number of LSLs, illustrated in Figure 4. To this end, we change the number of LSLs from 0 to 16, in increments of 2, and, for each point, we place an additional LSL on the bottom and on the top of the encoder, using the source and target languages to index them, respectively. For example, 4 LSLs corresponds to LSL (SRC=$\{1, 2\}$ & TGT=$\{15, 16\}$). We see that adding more LSLs helps performance, but only up to a point (in this case, 4 layers), and that afterwards, performance degrades, except for an outlier at 12 LSLs. This implies that while the language-specific layers boost performance, having shared layers is crucial for knowledge transfer.

F Per Language results

In Table 11, we show aggregate scores for each language group: European (DE, EN, ES, FR, IT, PT), CJK (ZH, JA, KO), and SW (isolated, since it is the only language in its family). Here, we see a similar trend, with our approach showing clear improvements both within groups, and between different groups.

```python
class LanguageSpecificEncoderLayer(nn.Module):
  def __init__(self, args, layer=0):
      super().__init__()
      self.index_language = args.language_specific_layers[layer]
      all_languages = sorted(set(self.get_lang(lp) for lp in args.lang_pairs))
      self.models = nn.ModuleDict({lang: TransformerEncoderLayer(args, layer) for lang in all_languages})

  def get_lang(self, lang_pair):
      # lang_pair is, for example: "en_US-de_DE"
      if self.index_language == "src":
          return lang_pair.split("-")[0]
      elif self.index_language == "tgt":
          return lang_pair.split("-")[1]
      else:
          raise ValueError(f"Invalid language `{self.index_language}`.")

  def forward(self, x, encoder_padding_mask, attn_mask: Optional[Tensor] = None):
      # self.lang_pair is set dynamically from outside the module.
      return self.models[self.get_lang(self.lang_pair)].forward(x, encoder_padding_mask, attn_mask)
```

Listing 1: Sample implementation of a Language-Specific Transformer Layer in FAIRSEQ.

	EN	ES	FR	IT	JA	KO	PT	SW	ZH
DE	213M	11.6M	36.6M	7.2M	1.2M	708K	5.4M	2.4M	1.8M
EN	–	230M	286M	96.3M	36.5M	2.3M	78.6M	708K	88.9M
ES	–	–	49.4M	14.9M	1.3M	772K	22.3M	6.9M	6.9M
FR	–	–	–	14.9M	1.2M	752K	12.9M	8M	25.3M
IT	–	–	–	–	736K	382K	7M	1.1M	964K
JA	–	–	–	–	–	511K	764K	820K	897K
KO	–	–	–	–	–	–	756K	536K	3M
PT	–	–	–	–	–	–	–	3.6M	1.1M
SW	–	–	–	–	–	–	–	–	962K

Table 7: Number of training sentence pairs for each language pair, after data de-duplication, cleaning, and down-sampling CCMatrix. We report only one language direction, as the data is the same for both directions.

	DE	EN	ES	FR	IT	JA	KO	PT	SW	ZH
DE →	–	67.4	52.1	61.0	54.2	30.1	26.1	59.0†	48.1	31.1
EN →	64.6	–	55.6	70.7	58.5†	35.0	28.8	70.6	55.4	34.9
ES →	53.1	59.2	–	57.7	52.6	28.0	23.4	54.7†	47.7	27.7
FR →	57.8	68.3	52.7	–	55.6	31.1	25.9	60.5†	50.1†	31.1
IT →	54.9	61.4	52.3	60.3	–	28.8	24.7	56.6	49.2	29.7
JA →	46.1	53.3	44.2	50.0	45.1	–	26.1	47.5	42.3	25.2
KO →	43.5	49.6	41.2	46.6	42.0	28.0	–	45.3	40.5	23.4
PT →	58.6	71.1	53.8†	64.0	55.5	30.5	26.4	–	53.1	31.8
SW →	47.0	57.3	43.6	51.0	44.5	23.0	20.5	50.0	–	23.9
ZH →	48.2	56.3	46.3	52.7	47.0	26.8	24.5	49.6	44.3	–

	DE	EN	ES	FR	IT	JA	KO	PT	SW	ZH
DE →	–	67.9	52.8	62.2	55.0	32.2	28.0	59.1†	49.2	32.0
EN →	65.1	–	56.2	71.2	58.6†	37.9	30.0	71.1	56.6	35.9
ES →	54.0	59.7	–	58.3	52.9	29.5	25.2	54.9†	49.1	29.1
FR →	58.6	68.9	53.0	–	56.2	32.0	28.0	60.8†	50.4†	32.1
IT →	55.7	61.7	53.0	60.7	–	29.9	26.4	57.1	50.6	30.2
JA →	47.9	55.0	45.1	50.6	45.9	–	27.3	48.6	44.4	26.7
KO →	45.4	52.1	42.9	48.2	43.9	30.7	–	47.2	43.2	25.6
PT →	59.5	71.5	54.0†	64.7	55.9	31.6	28.9	–	54.6	32.7
SW →	49.2	59.7	45.2	52.9	46.1	25.8	22.7	52.0	–	24.9
ZH →	49.7	57.1	47.3	53.9	47.9	29.1	26.5	50.0	46.5	–

Table 8: Comparison of the baseline model (*left*) and our learned architecture LSL-NAS with dense pre-training (*right*) for each language pair, on the Flores-101 test set. Our approach gives *significant* CHRF gains for most language pairs. Statistically *non-significant* improvements using paired bootstrap resampling are marked with † for $p \geq 0.05$ (in total 6 of the 90 language pairs).

	DE	EN	ES	FR	IT	JA	KO	PT	SW	ZH
DE →	–	67.4	50.9	60.6	53.6	31.6	26.2	58.5	48.3	30.3
EN →	64.0	–	55.4	70.8	58.4	35.7	29.0	70.0	55.5	33.3
ES →	52.4	59.7	–	57.6	52.6	28.5	24.0	54.2	48.4	27.8
FR →	57.6	68.8	52.3	–	55.5	30.6	25.5	60.0	50.1	29.7
IT →	54.3	61.8	51.4	59.7	–	29.6	24.0	55.8	49.1	29.1
JA →	46.0	54.0	43.5	49.3	45.4	–	26.4	47.2	42.4	25.2
KO →	43.1	50.1	40.6	45.9	42.2	27.8	–	44.7	40.3	22.7
PT →	57.9	71.2	53.1	63.8	54.9	31.0	26.9	–	52.5	30.9
SW →	47.2	58.4	43.7	50.9	44.8	23.9	20.7	50.3	–	23.9
ZH →	48.0	56.6	45.9	52.4	47.0	27.6	24.5	49.0	44.2	–

	DE	EN	ES	FR	IT	JA	KO	PT	SW	ZH
DE →	–	67.9	51.1	61.2	54.3	32.9	27.8	58.3	48.4	31.5
EN →	65.1	–	55.5	71.1	58.9	37.8	31.0	70.4	57.2	34.5
ES →	53.5	59.9	–	58.2	52.8	29.4	25.7	54.4	49.3	28.8
FR →	58.4	69.5	52.4	–	56.1	32.5	28.0	59.9	50.1	30.9
IT →	55.1	62.0	51.7	60.0	–	30.6	26.8	56.1	50.5	30.1
JA →	47.1	55.2	44.5	51.1	46.3	–	27.5	48.5	44.3	26.2
KO →	45.1	52.2	42.0	47.6	43.6	30.7	–	45.8	43.0	24.6
PT →	58.8	71.7	53.4	64.6	55.3	32.5	29.3	–	53.9	31.8
SW →	49.0	61.0	45.0	52.9	46.5	26.4	23.7	52.6	–	24.2
ZH →	49.2	57.7	46.4	53.3	47.7	29.8	26.3	50.1	46.4	–

Table 9: Comparison of the baseline model (*left*) and our learned architecture LSL-NAS with dense pre-training (*right*) for each language pair, on the Flores-101 validation set.

Model	CHRF	SPBLEU	$\lvert\boldsymbol{\theta}\rvert$	$\lvert\boldsymbol{\theta}_{\text{eff}}\rvert$
Separate Decoder Baseline	45.5	26.0	299M	186M
LSL-NAS	46.4	27.2	441M	186M
+ *Dense Pre-training*	**46.8**	**27.5**	441M	186M
Language Adapters ENC 128	45.8	26.4	321M	207M
+ hidden dim 640	46.2	26.9	426M	261M
Language Adapters ENC 256	45.7	26.3	342M	228M
+ hidden dim 640	46.3	27.1	452M	282M
Language Adapters ENC 512	45.6	26.3	384M	270M
Language Adapters ENC (SRC+TGT) 128	45.7	26.3	321M	207M
Language Adapters ENC (SRC+TGT) 256	46.1	26.7	342M	228M
Language Adapters ENC (SRC+TGT) 512	46.0	26.7	384M	270M
Language-Pair Adapters ENC 128	45.2	25.7	491M	207M
Language-Pair Adapters ENC 256	45.3	25.8	680M	228M
Language-Pair Adapters ENC 512	45.3	25.9	1057M	270M
Language Adapters ENC+DEC 256	45.5	26.0	350M	236M
Language Adapters ENC+DEC 512	46.0	26.4	400M	286M
Language Adapters ENC+DEC 768	46.1	26.6	449M	336M
Language Adapters ENC+DEC 1024	46.2	26.7	499M	385M

Table 10: Comparison of different Adapter Blocks configurations on the separate decoder architecture.

Direction	Sep. Decoder	Ours	Δ
EUR → EUR	59.1	59.7	**+0.6**
EUR → CJK	29.2	30.6	**+1.4**
EUR → SW	50.6	51.7	**+1.1**
CJK → EUR	47.4	48.8	**+1.4**
CJK → CJK	25.7	27.7	**+2.0**
CJK → SW	42.4	44.7	**+2.3**
SW → EUR	48.9	50.9	**+2.0**
SW → CJK	22.4	24.4	**+2.0**

Table 11: Comparison of LSL-NAS with pre-training compared to the separate baseline model per language family. Our approach gives substantial average CHRF gains for all, which are statistically significant for all but 6 of the 90 translation directions.

G COMET results

We show COMET, CHRF, and SPBLEU scores, averaged over all language pairs in Table 12. We show the scores for the baseline (i.e., non-LSL), our LSL model, and the best Adapter model for both the separate decoder and the shared decoder architectures. In all metrics, our proposed architectures outperform the remaining models.

H Inference Speed

We report the inference times for the various architectures we considered in Table 13. We report tokens/second on the DE-EN test set[9], averaged over 5 runs. Our latency measurements were collected using a single NVIDIA V100 GPU (Speed GPU) or a single-threaded Intel Xeon Platinum 8275CL CPU @ 3.00GHz (Speed CPU), both with batch size of 1, which faithfully captures the inference on a deployed neural machine translation model. As expected, the latency of shared decoder models is the same as that of similar separate decoder models (since only one of the decoders is used at inference time) so, for succinctness, we only report the separate decoder numbers.

A couple of comments regarding the Adapter models: 1) we do not report speed numbers for the "Language Adapters ENC (SRC+TGT)" as the architecture is the same as "Language Adapters ENC"; 2) inference speed does not change significantly when adding encoder adapters, but only when adding adapters to the decoder.

[9]We repeated these measurements for language pairs, such as EN-ZH, with similar results.

Model	COMET	CHRF	SPBLEU
Separate Decoder Baseline	0.45285	45.5	26.0
LSL-NAS	0.49577	46.4	27.2
+ *Dense Pre-training*	0.50759	46.8	27.5
Language Adapters ENC (SRC+TGT) 256	0.48265	46.1	26.7
Shared Decoder Baseline	0.36975	44.7	24.9
LSL-NAS-SD	0.46542	46.3	26.7
+ *Dense Pre-training*	0.48357	46.6	27.1
Shared Decoder Adapters 512	0.41849	45.3	25.6

Table 12: COMET, CHRF, and SPBLEU scores for the (non-LSL) baseline, our LSL models, and the best adapter model for the separate decoder and shared decoder architectures. These scores are averaged over all language pairs.

Architecture	Speed GPU	Speed CPU
Shared Decoder Baseline	195.2 ± 2.6	61.4 ± 0.3
Separate Decoder Baseline	194.3 ± 1.4	61.7 ± 0.2
+ hidden dim 640	191.9 ± 1.6	54.0 ± 0.2
+ hidden dim 704	189.8 ± 1.7	51.6 ± 0.3
+ hidden dim 768	187.7 ± 2.1	48.4 ± 0.2
Language Adapters ENC 128	188.1 ± 1.8	61.2 ± 0.3
Language Adapters ENC 256	186.0 ± 1.6	61.1 ± 0.3
Language Adapters ENC 512	187.6 ± 1.1	61.0 ± 0.2
Language Adapters ENC+DEC 256	165.2 ± 2.4	57.6 ± 0.3
Language Adapters ENC+DEC 512	165.1 ± 4.5	57.2 ± 0.2
Language Adapters ENC+DEC 768	164.4 ± 2.1	56.9 ± 0.3
LSL-NAS	195.0 ± 1.1	61.3 ± 0.2
LSL-NAS-SD	195.5 ± 4.7	61.4 ± 0.3

Table 13: Tokens/second comparison of different models on the Flores-101 DE-EN test set. We show the average over 5 runs, and the associated standard deviation. The latency of shared decoder models is the same as that of similar separate decoder models so, for succinctness, we only report the separate decoder numbers.